

📄 Apri Slide della Lezione

Settimana 3 — L'estate del 1654: contare i futuri possibili

Come due lettere tra Parigi e Tolosa hanno insegnato a pesare ciò che non è ancora accaduto, e perché lo stesso conteggio spiega le collisioni nelle tabelle hash.

Prima di cominciare: che cosa ci serve dalle Settimane 1 e 2

Questa settimana si **conta** molto, ma gli strumenti che servono sono pochi e già noti. Se uno di questi ti sembra incerto, rileggi la sezione indicata.

- **Spazio campionario equiprobabile.** Se Ω ha un numero finito di esiti tutti ugualmente plausibili, la probabilità di un evento A si riduce a un conteggio: $P(A) = |A|/|\Omega|$, dove $|A|$ è il numero di esiti dentro A (Settimana 1, §7). Dimenticare l'ipotesi «tutti ugualmente plausibili» è l'errore più comune: ci torneremo.
- **Complementare.** $P(A^c) = 1 - P(A)$ (Settimana 1, §6B). Spesso è più facile contare ciò che *non* vogliamo.
- **Regola del prodotto.** $P(A \cap B) = P(A) P(B | A)$ (Settimana 1, §8). Per esempio, due assi di fila da un mazzo di 52 carte, senza rimettere la prima: $\frac{4}{52} \cdot \frac{3}{51} = \frac{1}{221}$.
- **Indipendenza.** Se A e B sono indipendenti, $P(A \cap B) = P(A) P(B)$ (Settimana 1, §9).
- **Alberi di probabilità.** Ogni cammino dalla radice a una foglia è un possibile futuro; la sua probabilità è il prodotto dei numeri sui rami (Settimana 2, Atto II). Useremo la stessa convenzione grafica.

Prima una trappola: due monete

Prima di qualsiasi storia, un esperimento di trenta secondi. **Lanci una moneta onesta due volte. Qual è la probabilità di ottenere almeno una testa?** Fermati e scrivi la tua risposta prima di leggere oltre.

Ecco un ragionamento che sembra impeccabile. Si lancia la prima volta: se esce testa (T) si è già vinto, e il secondo lancio non serve. Se esce croce (C), si lancia la seconda volta, e può uscire testa o croce. Gli esiti da distinguere sono tre:

$$T, \quad CT, \quad CC,$$

e due su tre contengono una testa. Risposta: $\frac{2}{3}$.

È il ragionamento che **Jean Le Rond d'Alembert**, uno dei maggiori matematici del suo tempo, pubblicò nell'ottobre del 1754 nella voce *Croix ou pile* dell'*Encyclopédie* (vol. IV, pp. 512–513). Nel testo riconosce che la risposta che si trova «in tutti gli autori» è quella delle quattro combinazioni, cioè «3 contro 1», ma la mette in dubbio: «il secondo lancio non conta per nulla», quindi le combinazioni vere sarebbero tre e le probabilità «2 contro 1». Estende anche il ragionamento a tre lanci (testa; croce, testa; croce, croce, testa; croce, croce, croce), e ne ricava «3 contro 1» al posto del «7 contro 1»

standard, convinto che la cosa meriti «l'attenzione dei calcolatori». La risposta giusta è $\frac{3}{4}$ per due lanci (e $\frac{7}{8}$ per tre), e se hai risposto $\frac{2}{3}$ hai fatto il suo stesso ragionamento.

Dov'è l'errore? I tre esiti non hanno la stessa probabilità. Il caso « T » vale $\frac{1}{2}$ da solo, mentre CT e CC valgono $\frac{1}{4}$ ciascuno. Dire «due su tre» equivale a trattarli come se valessero tutti $\frac{1}{3}$. Il ragionamento si ripara in un modo solo: **si lanciano comunque entrambe le monete**, anche quando il secondo lancio, nella realtà, non servirebbe. Le sequenze diventano quattro,

$$TT, \quad TC, \quad CT, \quad CC,$$

tutte con probabilità $\frac{1}{4}$, e tre di esse contengono almeno una testa: $\frac{3}{4}$. Il caso « T » di prima era in realtà due sequenze fuse insieme, TT e TC .

Tieni a mente questa trappola, perché è il filo di tutta la settimana.



Una cautela storica

L'episodio è citato in moltissimi manuali come esempio tipico dell'errore sull'equiprobabilità. Non fu però una svista: d'Alembert conosceva la risposta standard e la contestò di proposito, nello stesso articolo in cui solleva anche il paradosso di San Pietroburgo. Il suo modello assegna probabilità uguali a casi *osservabili* (il gioco finisce con la prima testa), e questo è il punto in cui diverge dalla pratica corrente. Fu criticato più volte, per esempio da Laplace nell'*Essai philosophique*. Qui ci interessa l'errore in sé, che è istruttivo comunque.

Atto I: Il dramma storico — l'estate del 1654

Un problema che aspettava da 160 anni

Nella Settimana 1 hai già incontrato il *problème des partis*: due giocatori hanno puntato la stessa somma su un gioco a punti, la partita viene interrotta con il punteggio di 2 a 1, e bisogna dividere la posta. Là ci siamo fermati al risultato (3/4 contro 1/4). Questa settimana lo riapriamo, perché dietro quel risultato c'è tutto ciò che serve per contare.

Il problema è molto più vecchio del 1654. Nel 1494 Luca Pacioli, nella sua *Summa de arithmetica*, propone questo caso: due squadre giocano a palla fino a **60 punti** (ogni goal vale 10), la posta totale è di **10 ducati**, e il gioco si interrompe sul **50 a 20**. La sua regola è dividere **in proporzione ai punti già fatti**: $50 : 20 = 5 : 2$, cioè $\frac{5}{7}$ della posta (circa 7,14 ducati) a una squadra e $\frac{2}{7}$ (circa 2,86) all'altra. Lo presenta con sicurezza: gli altri pareri che ha incontrato, scrive, gli sembrano «incoerenti».

Una regola così semplice ha però un difetto evidente, che nel 1556 Niccolò Tartaglia mise in luce con un caso estremo. Se una squadra ha 10 punti e l'altra zero, la regola assegna **tutta** la posta a chi ha i 10 punti, anche se mancano ancora 50 punti alla vittoria: per lui una conclusione del genere non aveva senso. Il difetto è di principio: la regola di Pacioli guarda a **ciò che è già successo** e ignora quanto manca per finire. Tartaglia stesso, d'altra parte, concludeva che il problema si risolve più per via di giudizio che di calcolo, e proponeva una regola alternativa basata sul vantaggio rispetto alla lunghezza del gioco.

Il cavaliere e le sue due domande

In un secolo e mezzo nessuno trovò una risposta accettata. Poi, nel 1654, **Antoine Gombaud (1607–1684)**, noto come il *Cavaliere de Méré*, gentiluomo parigino e giocatore d'azzardo, pone a **Blaise Pascal (1623–1662)** due questioni che nascono dal tavolo da gioco: la divisione della posta in una partita interrotta (*parti des parties*, «divisione dei punti») e un problema sui dadi (*parti des dés*).

Pascal ne discute per lettera con **Pierre de Fermat (1601–1665)**, giurista e matematico a Tolosa. La prima lettera di Pascal è andata perduta; sopravvivono le altre, l'ultima datata 27 ottobre 1654, ristampate nelle *Œuvres de Fermat* (II, pp. 288–314, 1894). In quella del 29 luglio, Pascal scrive a Fermat di aver letto la sua soluzione e aggiunge, a proposito di de Méré: «*M. de Méré never could find the true value for the points nor a method to arrive to it*» («de Méré non riuscì mai a trovare il vero valore per i punti, né un metodo per arrivarci»). Nella stessa lettera compare una frase che racconta l'emozione dell'accordo: «*I see well that truth is the same in Toulouse as in Paris*» («vedo bene che la verità è la stessa a Tolosa e a Parigi»). Le citazioni sono dalla traduzione inglese delle lettere, la traduzione italiana è nostra.

In quella lettera Pascal racconta il «grande scandalo» di de Méré, che pensava di aver trovato una contraddizione nell'aritmetica. Due scommesse, entrambe del tipo «almeno uno»:

- **Scommessa A.** Si lancia **un dado quattro volte** e si punta che esca **almeno un sei**. De Méré la giocava da anni e *sapeva* che conviene: a lungo andare chi punta così vince. Con tre lanci invece non conviene: quattro è il minimo. Pascal la scrive così: le probabilità sono «**671 contro 625**». Significa che, tra tutte le possibili sequenze di quattro lanci (sono 1296), 671 contengono almeno un sei e 625 no; nell'Atto IV vedremo da dove nascono questi numeri.
- **Scommessa B.** Si lanciano **due dadi ventiquattro volte** e si punta che esca **almeno un doppio sei**. De Méré ragionava per proporzione. Con un dado le facce sono 6 e bastano 4 lanci. Con due dadi le combinazioni possibili sono 36, cioè **6 volte tante**, e quindi dovrebbero servire **6 volte tanti lanci**: $4 \cdot 6 = 24$. In altre parole, 24 sta a 36 come 4 sta a 6. Ma giocando, de Méré vedeva che questa scommessa **non** conveniva.

Per lui era un paradosso: la stessa proporzione, due risultati opposti. «L'aritmetica si contraddice», concludeva. Pascal risponde che la ragione si vede facilmente «dai principi di cui Fermat dispone». Lo vedremo anche noi, più avanti in questa settimana, con lo stesso trucco che useremo per i compleanni.

Quello che nasce da quelle lettere

Pascal e Fermat risolvono il problema dei punti con due metodi diversi, e la differenza tra i due sarà la prossima sezione: quello di Fermat consiste nel **contare tutti i futuri possibili**, quello di Pascal nel **dedurre ogni caso dal caso successivo**. Il metodo di Fermat diventa difficile da applicare quando i casi crescono, ed è proprio lì che nasce la *combinatoria* come strumento di calcolo: Pascal stesso scrive di aver trovato un'abbreviazione perché «la fatica delle combinazioni è eccessiva».

Pochi anni dopo, nel 1657, **Christiaan Huygens** pubblica *De ratiociniis in ludo aleae*, il primo trattato a stampa sul tema: lo incontreremo nella Settimana 4. Nel 1713, otto anni dopo la morte del suo autore, esce l'*Ars Conjectandi* di **Jacob Bernoulli** (scansione su Internet Archive), che ordina permutazioni e combinazioni in una teoria sistematica e le presenta come l'arte di *congetturare*, cioè di ragionare quando non si può essere certi.

? Le domande che ci guideranno

1. Perché dividere la posta in proporzione ai punti fatti è sbagliato, e che cosa bisogna pesare al suo posto?
2. Come si fa a **contare** i futuri possibili senza elencarli uno a uno quando diventano migliaia?
3. Dove sta l'errore di de Méré, e perché «ventiquattro su trentasei» non è «quattro su sei»?
4. Se contare i futuri è così potente, che cosa ci dice sui sistemi che *noi* costruiamo, come le tabelle hash?

Atto II: Pesare i futuri possibili – i due metodi del 1654

L'albero delle manche

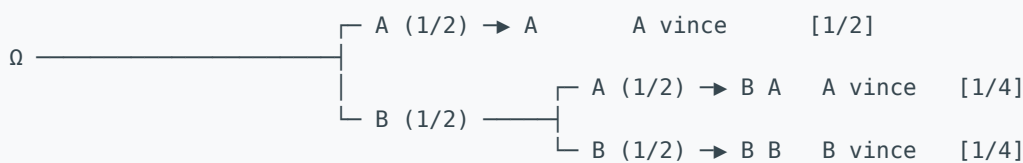
Riprendiamo il caso della Settimana 1. Si gioca a **3 punti**, ciascuno ha puntato 32 pistole (la «pistola» era una moneta d'oro dell'epoca; posta totale: 64). Il giocatore *A* ha 2 punti, *B* ne ha 1: ad *A* manca **un** punto, a *B* ne mancano **due**. Prima di disegnare l'albero fissiamo con precisione che cosa stiamo assumendo, come nelle lettere. Sono **due** ipotesi, e una sola non basta.

1. **Uguale bravura.** In ogni manche *A* vince con probabilità $\frac{1}{2}$ e *B* con probabilità $\frac{1}{2}$.
2. **Indipendenza delle manche.** L'esito di una manche non cambia le probabilità delle manche successive. Se *A* ha appena vinto, la manche dopo è ancora $\frac{1}{2}$ a $\frac{1}{2}$: niente «serie fortunata», niente stanchezza, niente rivincita.

La seconda ipotesi è ciò che ci permette di scrivere $\frac{1}{2}$ su **ogni** ramo dell'albero, anche dopo che qualcosa è già successo, e di moltiplicare lungo i cammini: è la regola del prodotto della Settimana 1, con $P(B | A) = P(B)$. Ha anche una conseguenza che useremo più avanti: **per sapere come andrà a finire la partita conta solo quanto manca a ciascuno, non la storia che ci ha portato fin lì**. Due partite in cui ad *A* mancano 2 punti e a *B* ne mancano 3 hanno lo stesso futuro, anche se una è arrivata a quel punto dopo *AAB* e l'altra dopo *ABA*.

Sono ipotesi del modello, non fatti: in un gioco reale potrebbero essere false. Pascal e Fermat le ammettono, e noi con loro.

Si gioca una manche alla volta e ci si ferma appena uno dei due raggiunge il traguardo. Se *A* vince la prossima manche, la partita è finita; se la vince *B*, se ne gioca un'altra, e a quel punto basta che *A* vinca per chiudere, altrimenti vince *B*:



È l'albero della Settimana 2, con la stessa convenzione: le parentesi tonde sono le probabilità dei rami, le quadre quelle delle foglie, che si ottengono moltiplicando lungo il cammino. Ogni foglia è un **futuro possibile**, e *A* vince in due di essi: $P(A \text{ vince}) = \frac{1}{2} + \frac{1}{4} = \frac{3}{4}$, $P(B \text{ vince}) = \frac{1}{4}$. Attenzione: $\frac{1}{2}$ è la probabilità che *A* vinca la *prossima* manche, ma *A* può vincere la partita anche perdendo la prossima, e all' $\frac{1}{2}$ va aggiunto il $\frac{1}{4}$ del futuro *BA*. La posta si divide allo stesso modo: **48 pistole ad A e 16 a B**.

Il principio, da fissare, è questo: **la posta si divide secondo le probabilità di vittoria, e queste si ottengono sommando i futuri possibili, non guardando il passato.** Il passato conta solo perché decide *quanto manca* a ciascuno.

Con pochi punti mancanti l'albero si disegna a mano. Ma proviamo un caso più grande, cambiando le regole del gioco: si gioca a **5 punti**, e la partita viene interrotta sul **2 a 0** per *A*. Ad *A* mancano allora $5 - 2 = 3$ manche, a *B* ne mancano $5 - 0 = 5$. Ammettiamo ancora che a ogni manche ciascuno vinca con probabilità $\frac{1}{2}$.

Il 2-0 è già stato giocato e non si cambia: l'albero descrive solo le manche **che restano da giocare**. Quante possono essere?

- **Al minimo 3 manche.** È il caso in cui *A* vince di fila le 3 manche che gli mancano (*AAA*). Meno di 3 non si può: ad *A* ne servono 3, e a *B* ne servirebbero 5.
- **Al massimo 7 manche.** Finché la partita non è decisa, tra le manche giocate dopo l'interruzione *A* ne ha vinte al più 2 (con 3 avrebbe già raggiunto il traguardo) e *B* al più 4 (con 5 avrebbe già finito). Dopo 6 manche si può quindi essere ancora indecisi, per esempio con *A* a 2 vittorie e *B* a 4. Ma la settima manche decide per forza: o *A* arriva a 3 o *B* arriva a 5. In generale, se a un certo punto della partita ad *A* mancano a manche per vincere e a *B* ne mancano b , da quel punto in poi si giocano al più $a + b - 1$ manche. Qui $a + b - 1 = 3 + 5 - 1 = 7$.

Le foglie dell'albero sono tutte le sequenze di manche che si fermano **nel momento in cui uno dei due raggiunge il traguardo**, e sono **56**: 35 con *A* vincente e 21 con *B* vincente. Vediamo da dove vengono questi numeri, contando le foglie una lunghezza alla volta. Scriviamo ogni sequenza dopo l'interruzione come una parola: *ABAA* significa che la prima manche dopo l'interruzione la vince *A*, la seconda *B*, la terza e la quarta *A*.

Le foglie in cui vince A. Dopo l'interruzione, ad *A* servono 3 vittorie. Se *A* vince la partita alla manche numero d , devono valere due cose:

1. l'**ultima** manche è sua: se avesse già raggiunto 3 vittorie prima, la partita sarebbe finita prima;
2. nelle manche **precedenti** ha vinto esattamente 2 volte, e tutte le altre manche le ha vinte *B*.

Contiamo ora le sequenze, una lunghezza alla volta.

- $d = 3$. Le manche precedenti sono 2 e *A* le vince entrambe: l'unica sequenza è *AAA*. **1 sequenza.**
- $d = 4$. Le manche precedenti sono 3: due le vince *A* e una *B*. Resta solo da decidere *in quale delle tre* ha vinto *B*, e le possibilità sono *BAA*, *ABA*, *AAB*. Aggiungendo in fondo la vittoria finale di *A* si ottengono *BAAA*, *ABAA*, *AABA*. **3 sequenze.**
- $d = 5$. Le manche precedenti sono 4: due di *A* e due di *B*. Scriviamo tutti i modi di disporle: *AABB*, *ABAB*, *ABBA*, *BAAB*, *BABA*, *BBAA*. Sono 6, e a ciascuna si aggiunge in fondo la *A* finale. **6 sequenze.**
- $d = 6$ e $d = 7$. Il ragionamento è identico: si dispongono 2 vittorie di *A* tra le 5 manche precedenti (per $d = 6$), o tra le 6 (per $d = 7$), e *B* vince le altre. Elencarle diventa lungo, e per questo il conteggio è già un problema di combinatoria. Il risultato è **10 e 15** sequenze. (In nessun caso *B* arriva a 5 vittorie prima di *A*: ne ha al massimo 4.)

Sommando: $1 + 3 + 6 + 10 + 15 = \mathbf{35}$ foglie con *A* vincente.

Le foglie in cui vince B. Il ragionamento è lo stesso, a parti invertite. A *B* servono 5 vittorie, quindi può vincere solo alla manche $d = 5, 6$ o 7 . L'ultima manche è sua, e nelle precedenti ha vinto esattamente 4 volte; le altre le ha vinte *A*, che quindi ha al massimo 2 vittorie e non ha ancora finito.

- $d = 5$. Le manche precedenti sono 4 e le vince tutte *B*: l'unica sequenza è *BBBBB*. **1 sequenza.**

- $d = 6$. Tra le 5 manche precedenti ne vince 4 B e una A : basta scegliere *quale* delle 5 è di A . **5 sequenze**.
- $d = 7$. Tra le 6 manche precedenti ne vincono 4 B e 2 A : sono **15** sequenze, lo stesso numero del caso $d = 7$ di prima.

Sommando: $1 + 5 + 15 = \mathbf{21}$ foglie con B vincente, e in tutto $35 + 21 = 56$ foglie. (Contare questi «posti» in modo sistematico, senza elencare, è proprio il tema dell'Atto III.)

Si noti che ogni foglia a profondità d (cioè dopo d manche giocate dall'interruzione) ha probabilità $(\frac{1}{2})^d$, perché è il prodotto di d rami da $\frac{1}{2}$. Le foglie con A vincente sono così distribuite:

manche giocate dall'interruzione (d)	foglie con A vincente	probabilità di ciascuna	contributo
3	1	$\frac{1}{8}$	$\frac{16}{128}$
4	3	$\frac{1}{16}$	$\frac{24}{128}$
5	6	$\frac{1}{32}$	$\frac{24}{128}$
6	10	$\frac{1}{64}$	$\frac{20}{128}$
7	15	$\frac{1}{128}$	$\frac{15}{128}$

Sommando: $P(A \text{ vince}) = \frac{16+24+24+20+15}{128} = \frac{99}{128}$. Funziona, ma per costruire la tabella abbiamo dovuto sapere **quanti** cammini finiscono in ciascuna riga (1, 3, 6, 10, 15), e questo è già un problema di conteggio. Nel 1654 Pascal e Fermat propongono due modi per aggirarlo.

Pascal: dedurre ogni caso dal successivo

Pascal, nella lettera del 29 luglio 1654, usa un metodo che chiama più «breve». Lo seguiamo un passo alla volta.

Passo 1: un nome per «quanto manca». Per l'indipendenza delle manche, il futuro della partita dipende solo da quanti punti mancano ad A e a B . Diamo allora un nome alla probabilità che A vinca: $V(a, b)$, quando ad A mancano a punti e a B ne mancano b . Non è un numero ma una **tabella**: a ogni coppia (a, b) corrisponde il suo valore. Di questi valori ne conosci già due, uno per ciascuno dei due giochi visti sopra. Nel gioco a 3 punti interrotto sul 2-1 ad A manca 1 punto e a B ne mancano 2, e l'albero piccolo ci ha dato $V(1, 2) = \frac{3}{4}$. Nel gioco a 5 punti interrotto sul 2-0 ad A mancano 3 punti e a B ne mancano 5, e le 56 foglie ci hanno dato $V(3, 5) = \frac{99}{128}$. Lo stesso simbolo V vale per entrambi i giochi, perché conta solo quanto manca a ciascuno, non il punteggio a cui si gioca.

Passo 2: i casi che non richiedono calcoli. Se $a = 0$, ad A non manca più nulla e ha già vinto: $V(0, b) = 1$. Se $b = 0$ ha già vinto B , quindi A ha perso: $V(a, 0) = 0$.

Passo 3: giochiamo una sola manche, sul caso piccolo. Riprendi l'albero del 2-1, cioè $(a, b) = (1, 2)$. La prossima manche ha due esiti:

- vince A , con probabilità $\frac{1}{2}$: ora mancano $(0, 2)$, la partita è finita e A ha vinto;
- vince B , con probabilità $\frac{1}{2}$: ora mancano $(1, 1)$. Guarda la parte di albero che parte da qui, il ramo B con le sue due foglie: è l'**albero di una partita nuova**, in cui a ciascuno manca un punto, cioè proprio lo stato $(1, 1)$.

Passo 4: perché il ramo B vale $V(1, 1)$. Dopo che B ha vinto la manche, la probabilità che A vinca la partita è quella di una partita che comincia da $(1, 1)$, perché per l'indipendenza il passato non conta: quel valore è $V(1, 1)$. La

probabilità che la partita passi dal ramo B e finisca con la vittoria di A è allora la probabilità di arrivare al ramo $(\frac{1}{2})$ per la probabilità di vincere da lì ($V(1, 1)$): è la regola del prodotto, $P(\text{ramo}) \cdot P(A \text{ vince} \mid \text{ramo})$. Sommando i due rami:

$$V(1, 2) = \frac{1}{2} V(0, 2) + \frac{1}{2} V(1, 1).$$

Passo 5: lo stesso ragionamento vale per ogni (a, b) . Se $a \geq 1$ e $b \geq 1$, giocando **una** manche si va in $(a - 1, b)$ se vince A , oppure in $(a, b - 1)$ se vince B , ciascuno con probabilità $\frac{1}{2}$. Quindi

$$V(a, b) = \frac{1}{2} V(a - 1, b) + \frac{1}{2} V(a, b - 1).$$

In parole: la probabilità di vincere da uno stato è la media delle probabilità di vincere dai due stati che si possono raggiungere con una manche. La formula riduce ogni stato a due stati più piccoli, finché si arriva ai casi del Passo 2, che sono noti. Ripercorriamo i tre casi della lettera, tutti nel gioco a 3 punti con posta totale di 64 pistole. Per dividere la posta in modo giusto, ciascuno deve ricevere una parte proporzionale alla sua probabilità di vincere:

- **Sul 2-1** ($a = 1, b = 2$): $V(1, 2) = \frac{1}{2} V(0, 2) + \frac{1}{2} V(1, 1) = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot \frac{1}{2} = \frac{3}{4}$, perché $V(1, 1) = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 0 = \frac{1}{2}$. Con questa proporzione, volendo ripartire la posta in modo giusto, su 64 pistole ne spettano **48** ad A .
- **Sul 2-0** ($a = 1, b = 3$): $V(1, 3) = \frac{1}{2} + \frac{1}{2} V(1, 2) = \frac{1}{2} + \frac{1}{2} \cdot \frac{3}{4} = \frac{7}{8}$, cioè **56** pistole ad A .
- **Sul 1-0** ($a = 2, b = 3$): $V(2, 3) = \frac{1}{2} V(1, 3) + \frac{1}{2} V(2, 2) = \frac{1}{2} \cdot \frac{7}{8} + \frac{1}{2} \cdot \frac{1}{2} = \frac{11}{16}$, cioè **44** pistole ad A e 20 a B .

Sono esattamente i numeri 48, 56 e 44 che Pascal scrive a Fermat nella lettera. Continuando a riempire la tabella dal basso si arriva anche al caso $a = 3, b = 5$ (il gioco a 5 punti interrotto sul 2-0 di prima):

	$b = 1$	$b = 2$	$b = 3$	$b = 4$	$b = 5$
$a = 1$	$\frac{1}{2}$	$\frac{3}{4}$	$\frac{7}{8}$	$\frac{15}{16}$	$\frac{31}{32}$
$a = 2$	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{11}{16}$	$\frac{13}{16}$	$\frac{57}{64}$
$a = 3$	$\frac{1}{8}$	$\frac{5}{16}$	$\frac{1}{2}$	$\frac{21}{32}$	$\frac{99}{128}$

L'ultima casella è il $\frac{99}{128}$ trovato prima sommando le foglie con A vincente. Con la tecnica di Pascal lo si ottiene con **15 calcoli**, uno per casella, invece di sommare decine di cammini. Il motivo è che nell'albero lo stesso stato compare più volte (dopo AB e dopo BA ci si trova nello stesso stato), e Pascal lo calcola una volta sola. In Informatica lo riconosci: è **programmazione dinamica**, e infatti il calcolo di $V(a, b)$ si scrive come una funzione ricorsiva con memoria (lo vedi nello Script 1).

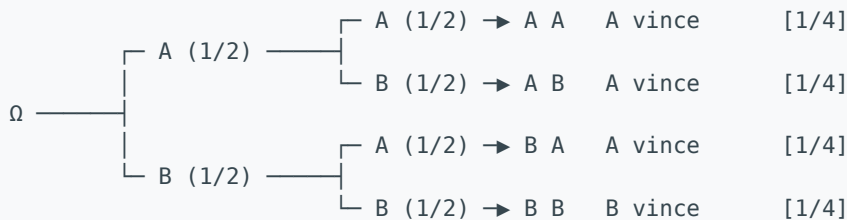
Fermat: contare tutti i futuri

Il metodo di Fermat, che conosciamo dalla lettera di Pascal (la sua lettera originale è andata perduta), parte da una difficoltà dell'albero reale: **non sappiamo quante manche si giocheranno**. A volte la partita finisce subito, a volte dopo molte manche, e le foglie hanno probabilità diverse tra loro. L'idea di Fermat è **non aspettare** che la partita finisca

davvero: si immagina di giocare *comunque* un numero fisso di manche, anche quelle che nella realtà non si giocherebbero.

Quante manche bastano? Se ad A mancano a punti e a B ne mancano b , dopo $n = a + b - 1$ manche la partita è certamente decisa. Vediamolo prima su un caso piccolo: il gioco a **3 punti**, interrotto sul **2-1** per A (ad A manca 1 punto, a B ne mancano 2, quindi $a = 1$ e $b = 2$). Qui $n = 1 + 2 - 1 = 2$. Infatti in 2 manche non è possibile che A ne vinca *meno* di 1 e insieme B ne vinca *meno* di 2: se A non ne vince nessuna le vince tutte e due B , e B arriva a 2. In generale, in $n = a + b - 1$ manche o A ne vince almeno a , oppure B ne vince almeno b , mai tutte e due.

Il caso piccolo: gioco a 3 punti, sul 2-1 ($a = 1, b = 2, n = 2$). Giochiamo sempre 2 manche:



Leggiamo l'albero. I futuri sono quattro, AA, AB, BA, BB , e hanno tutti la stessa probabilità $\frac{1}{4}$. In tre di essi, AA, AB e BA , A vince almeno 1 manche e quindi vince la partita; solo in BB vince B . Dunque $P(A \text{ vince}) = \frac{3}{4}$, lo stesso risultato dell'albero reale.

Confrontiamo con l'albero reale, che aveva tre foglie: A con probabilità $\frac{1}{2}$, BA con $\frac{1}{4}$ e BB con $\frac{1}{4}$. Le foglie BA e BB ci sono anche qui, identiche. Cambia solo la foglia A , che qui diventa due futuri, AA e AB , ciascuno di probabilità $\frac{1}{4}$, e $\frac{1}{4} + \frac{1}{4} = \frac{1}{2}$ è proprio la probabilità della foglia A di prima. Perché si spezza in due? Nella realtà, dopo che A ha vinto la prima manche, la partita è finita e la seconda manche **non si gioca**. Con Fermat la giochiamo comunque, solo per immaginazione: è una manche **fittizia**, perché non influisce sul risultato. Qualunque ne sia l'esito (A oppure B), A aveva già raggiunto il traguardo, e due esiti di probabilità $\frac{1}{2}$ e $\frac{1}{2}$ sommano a 1: la probabilità della foglia originale non cambia. Per questo aggiungere manche fittizie non modifica la probabilità che A vinca, e permette di far arrivare tutti i futuri alla stessa lunghezza.

Perché basta contare. Tutti i futuri hanno la stessa lunghezza n , e per l'indipendenza delle manche ognuno ha la stessa probabilità $(\frac{1}{2})^n$: è il prodotto di n rami da $\frac{1}{2}$. Quindi la probabilità che A vinca è semplicemente

$$P(A \text{ vince}) = \frac{\text{futuri in cui } A \text{ vince almeno } a \text{ manche}}{2^n},$$

e il problema diventa un conteggio: sul 2-1 sono $3/2^2 = \frac{3}{4}$. Nell'albero reale, invece, le foglie avevano probabilità diverse ($\frac{1}{2}, \frac{1}{4}, \frac{1}{4}$) e bisognava pesarle una per una.

Un secondo caso: gioco a 3 punti, sull'1-0 ($a = 2, b = 3, n = 4$). Si gioca ancora a 3 punti, ma ora il punteggio è 1-0 per A : ad A mancano 2 punti e a B ne mancano 3. Giochiamo sempre $n = 2 + 3 - 1 = 4$ manche, quindi i futuri sono $2^4 = 16$, tutti con probabilità $\frac{1}{16}$. A vince quando ne vince **almeno 2**. Con soli 16 futuri possiamo scriverli tutti, come parole di 4 lettere (la i -esima lettera dice chi vince la i -esima manche) e raggrupparli per **quante manche vince A** , che chiamiamo k :

k (vittorie di A)	futuri	quanti
0	$BBBB$	1
1	$ABBB, BABB, BBAB, BBBA$	4
2	$AABB, ABAB, ABBA, BAAB, BABA, BBAA$	6
3	$AAAB, AABA, ABAA, BAAA$	4
4	$AAAA$	1

Controllo: $1 + 4 + 6 + 4 + 1 = 16$, e cioè ci sono tutti. Nelle righe con $k = 2, 3, 4$ A vince, quindi i futuri favorevoli sono $6 + 4 + 1 = 11$ e

$$P(A \text{ vince}) = \frac{11}{16},$$

lo stesso valore trovato con Pascal.

Il caso grande: gioco a 5 punti, sul 2-0 ($a = 3, b = 5, n = 7$). È la partita di prima, quella con 56 foglie e la tabella di Pascal. Con Fermat si giocano sempre $n = 3 + 5 - 1 = 7$ manche, anche quelle che nella realtà non servirebbero: i futuri sono $2^7 = 128$, tutti con probabilità $\frac{1}{128}$, e A vince quando ne vince **almeno 3**. Qui elencarli tutti sarebbe troppo lungo, ma il raggruppamento per k è lo stesso della tabella di prima: i futuri con esattamente k vittorie di A sono 1, 7, 21, 35, 35, 21, 7, 1 per $k = 0, \dots, 7$ (come si contano senza elencarli lo capiremo nell'Atto III), quindi

$$P(A \text{ vince}) = \frac{35 + 35 + 21 + 7 + 1}{128} = \frac{99}{128},$$

lo stesso risultato di prima, senza costruire la tabella per righe e senza sapere nulla dei cammini che finiscono in anticipo.

Quando conviene Fermat, quando Pascal

I due metodi calcolano lo stesso numero. Per **calcolare** con un programma, la ricorsione di Pascal è più economica: servono $a \cdot b$ stati, mentre elencare i futuri di Fermat significa passare in rassegna 2^n sequenze, che con $n = 4$ sono 16, con 10 sono 1024, con 30 oltre un miliardo. È il limite che Pascal nota subito nella lettera del 29 luglio: «*the labor of combinations is excessive*» («la fatica delle combinazioni è eccessiva»).

Ma il valore del metodo di Fermat non sta nell'elencare: **non serve elencare i futuri, serve sapere quanti sono** quelli in cui A vince, e questo si scrive in una sola formula con i numeri 1, 7, 21, 35, ... Fermat è il metodo che ci porta ai coefficienti binomiali, Pascal è il metodo che ci dà l'algoritmo. Sono lo stesso conto, e imparare a contare senza elencare è l'argomento dell'Atto III.

Esplora tu stesso: i futuri del problema dei punti

Scegli quanti punti mancano ad A e a B . Il widget elenca i futuri (« A » e « B » indicano chi vince ciascuna manche fittizia), colorando di blu quelli in cui vince A e di arancione quelli in cui vince B . Prova $a = 1, b = 2$ (il caso di

prima), poi $a = 3, b = 3$: quanti futuri sono? E come cambia la divisione della posta?

Mancano ad A: punti Mancano a B: punti

BBBBB	BBBBA	BBBAB	BBBAA	BBABB	BBABA	BBAAB	BBAAA	BABBB	BABBA	BABAB
BABAA	BAABB	BAABA	BAAAB	BAAAA	ABBBB	ABBBA	ABBAB	ABBAA	ABABB	ABABA
ABAAB	ABAAA	AABBB	AABBA	AABAB	AABAA	AAABB	AAABA	AAAAB	AAAAA	

Si giocano comunque 5 manche fittizie: 32 futuri equiprobabili. A vince in 16, B in 16. $P(A \text{ vince}) = 16/32 = 0.5000$. Su una posta di 64 pistole: A ne riceve 32.00, B 32.00.

Script 1 – Pascal, Fermat e il Monte Carlo a confronto

Tre modi di calcolare la stessa probabilità: la ricorsione di Pascal (`pascal`), l'enumerazione dei futuri di Fermat (`fermat`) e la simulazione di molte partite (`montecarlo`). Devono dare lo stesso risultato, e i tre casi della lettera del 29 luglio. L'ultima riga mostra che il metodo non dipende dal fatto che i giocatori siano ugualmente bravi: basta cambiare p .

Blocco di codice di 49 righe omissso dalla versione PDF: lo trovi, e puoi eseguirlo direttamente nel browser, nella versione online del capitolo.

Atteso: 0.7500, 0.8750, 0.6875 con tutti e tre i metodi (il Monte Carlo a meno di qualche millesimo), e 0.5556 per il caso $p = 1/3$.

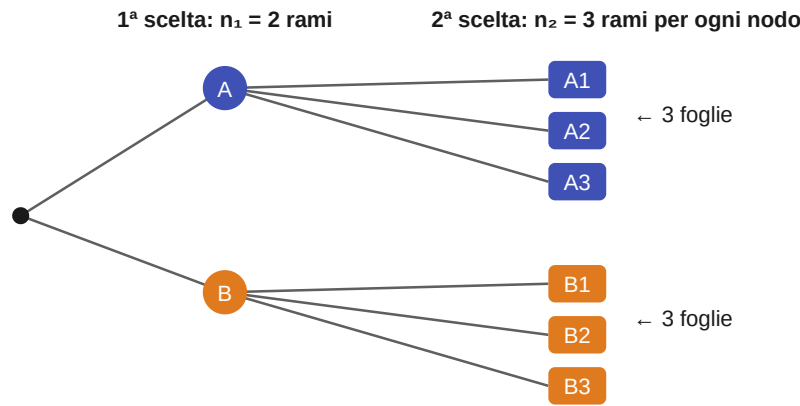
Atto III: Contare senza elencare – la combinatoria minima

Fermat elencava i futuri, e funzionava finché erano 4 o 16. Ma 2^{30} futuri non si elencano: bisogna imparare a **contarli**. La combinatoria che ci serve è poca, e la costruiamo con tre idee, ciascuna conseguenza della precedente. Niente urne, biglie o anagrammi: ci fermiamo non appena ne abbiamo abbastanza per capire il coefficiente binomiale e per arrivare alle tabelle hash.

Prima idea: il principio della moltiplicazione

Se una scelta si può fare in n_1 modi e, **qualunque sia** stata, la scelta successiva si può fare in n_2 modi, le due scelte insieme si possono fare in $n_1 \cdot n_2$ modi.

Lo si vede con l'albero della Settimana 2, senza le probabilità. Ogni **foglia** è un percorso completo dalla radice, cioè una coppia di scelte. Dalla radice partono n_1 rami; da *ciascuno* dei nodi raggiunti ne partono altri n_2 . Le foglie sono quindi n_1 gruppi da n_2 foglie ciascuno: in tutto $n_1 \cdot n_2$.



foglie totali = $2 \cdot 3 = 6$ (ogni foglia è un percorso completo: una coppia di scelte)

Due scelte: 2 modi per la prima (A o B), poi 3 modi per la seconda, qualunque sia stata la prima. Il numero di foglie è il prodotto del numero di rami di ciascun livello.

Con più scelte si prosegue allo stesso modo: il numero di foglie è il prodotto dei rami di ogni livello, $n_1 \cdot n_2 \cdot \dots \cdot n_k$. Se tutti i livelli hanno b rami, diventa b^k .

Per esempio, una password di 8 caratteri scelti tra 62 simboli (26 minuscole, 26 maiuscole, 10 cifre) si può scrivere in $62^8 \approx 2,2 \cdot 10^{14}$ modi. In generale, **sequenze di lunghezza k su n simboli, con ripetizione, sono n^k** : tre maniche con due esiti ciascuna danno $2^3 = 8$ futuri, quindici bit danno 2^{15} stringhe.

*Nota bene: il numero di rami può **cambiare da un livello al successivo** ($n_1 \neq n_2$), ma al **livello successivo deve essere lo stesso per ogni nodo**, cioè non deve dipendere da quale ramo è stato preso prima. Se dipendesse dal percorso (per esempio 3 rami dopo A ma solo 2 dopo B), il prodotto non vale più: bisogna contare le foglie di ogni sottoalbero e **sommarle** ($3 + 2 = 5$, non $2 \cdot 3$).*

Seconda idea: le disposizioni e il fattoriale

Nella prima idea ogni livello dell'albero offriva sempre gli stessi n simboli: una password può ripetere un carattere. Ma a volte la ripetizione è vietata: se assegniamo 3 server diversi a 3 compiti, un server già usato non può essere scelto di nuovo. Vediamo che cosa cambia nell'albero, con un esempio minuscolo: scegliamo 2 lettere **diverse** tra $\{A, B, C\}$, una dopo l'altra.

- Al primo livello i rami sono 3: A, B o C .
- Al secondo livello, da ciascun nodo, i rami sono 2: la lettera appena scelta non si può ripetere, restano le altre due.

Attenzione a un dettaglio che riguarda la nota di prima. Le lettere offerte *dipendono* dal ramo scelto (dopo A restano B, C ; dopo B restano A, C), ma il loro **numero** no: è sempre 2. La nota chiedeva proprio questo, quindi il prodotto vale ancora: $3 \cdot 2 = 6$ foglie, cioè

AB, AC, BA, BC, CA, CB .

Guarda questa lista: AB e BA usano le stesse due lettere, ma sono **due foglie diverse**, perché sono due percorsi diversi nell'albero (prima A poi B , oppure prima B poi A). Quando diciamo che «l'ordine conta» intendiamo

esattamente questo: **due sequenze che contengono gli stessi oggetti in ordine diverso sono contate come due risultati distinti**. È una conseguenza naturale di come abbiamo costruito l'albero (una foglia = un percorso), non una regola in più. Lo scopo di questa idea è contare *tutte* le sequenze, ordine compreso; nella prossima vedremo come si fa se invece vogliamo che AB e BA valgano come un unico risultato.

Generalizziamo: scegliamo k oggetti **tutti diversi** tra n , formando una sequenza. Il primo si sceglie in n modi, il secondo in $n - 1$ (uno è già preso), il terzo in $n - 2$, e così via fino al k -esimo, che si sceglie in $n - k + 1$ modi. Per la moltiplicazione:

$$D(n, k) = n(n-1)(n-2) \cdots (n-k+1) = \frac{n!}{(n-k)!}.$$

Qui $n! = n(n-1) \cdots 2 \cdot 1$ è il **fattoriale** ($0! = 1$ per convenzione). Il caso $k = n$ dà $n!$: i modi di **ordinare** n oggetti distinti. Questo è il motivo per cui nessun algoritmo di ordinamento può fare a meno di circa $n \log n$ confronti: dovendo distinguere $n!$ ordinamenti con risposte sì/no servono almeno $\log_2(n!)$ domande (lo vedrai nel corso di Algoritmi).

Una situazione che ci servirà tra poco: k chiavi vanno in una tabella con m slot, **ognuna in uno slot diverso**. La prima ha m scelte, la seconda $m - 1$, la terza $m - 2$... Il numero di modi è proprio $D(m, k) = m(m-1) \cdots (m-k+1)$.

Terza idea: le combinazioni

Spesso però AB e BA devono valere come un solo risultato: scegliamo 3 server su 10 per un rilascio sperimentale, e non ci interessa *in che ordine* li abbiamo nominati, conta solo *quali* sono. Partiamo da ciò che sappiamo contare, le sequenze, dove l'ordine distingue: $D(10, 3) = 720$. Ma ogni gruppo di 3 server compare in $3! = 6$ ordini diversi, e li abbiamo contati tutti e sei. Per contare i **gruppi** dobbiamo dividere per 6. In generale, un sottoinsieme di k elementi compare $k!$ volte tra le sequenze, quindi:

Coefficiente binomiale

$$\binom{n}{k} = \frac{D(n, k)}{k!} = \frac{n!}{k!(n-k)!}$$

è il numero di modi di **scegliere un sottoinsieme di k elementi** da un insieme di n , **ignorando l'ordine**. Si legge « n su k ».

Con i server: $\binom{10}{3} = \frac{720}{6} = 120$. Due proprietà si vedono subito dalla definizione: $\binom{n}{k} = \binom{n}{n-k}$ (scegliere chi *entra* o chi *resta fuori* è lo stesso), e il caso $k = 2$, $\binom{n}{2} = \frac{n(n-1)}{2}$, è il **numero di coppie** in un gruppo di n persone. Quest'ultima ci tornerà utilissima nell'Atto IV.

Il triangolo di Pascal e i futuri di Fermat

Finora abbiamo calcolato $\binom{n}{k}$ con i fattoriali, che diventano enormi in fretta. Esiste un modo più economico: costruire ogni coefficiente a partire da quelli più piccoli, usando solo somme. Per scoprire come, partiamo da un esempio che si può elencare a mano.

Abbiamo 4 server, $\{A, B, C, D\}$, e ne scegliamo 2. Sappiamo già che i gruppi sono $\binom{4}{2} = \frac{4 \cdot 3}{2} = 6$. Eccoli:

$$AB, AC, AD, BC, BD, CD.$$

Ora fissiamo l'attenzione su un server in particolare, A , e dividiamo i gruppi in due famiglie, a seconda che A ci sia o no:

- **con** A : AB, AC, AD . Sono 3. Una volta messo A nel gruppo, ne manca ancora 1 da scegliere tra gli altri 3 server (B, C, D): $\binom{3}{1} = 3$.
- **senza** A : BC, BD, CD . Sono 3. Qui A è escluso, quindi i 2 posti si riempiono scegliendo tra gli stessi 3 server: $\binom{3}{2} = 3$.

Ogni gruppo sta in una famiglia e in una sola, perché A o c'è o non c'è. Quindi $\binom{4}{2} = \binom{3}{1} + \binom{3}{2} = 3 + 3 = 6$.

Il ragionamento non dipende dai numeri. Prendiamo un insieme di n elementi, ne fissiamo uno, x , e contiamo i sottoinsiemi di k elementi in due famiglie:

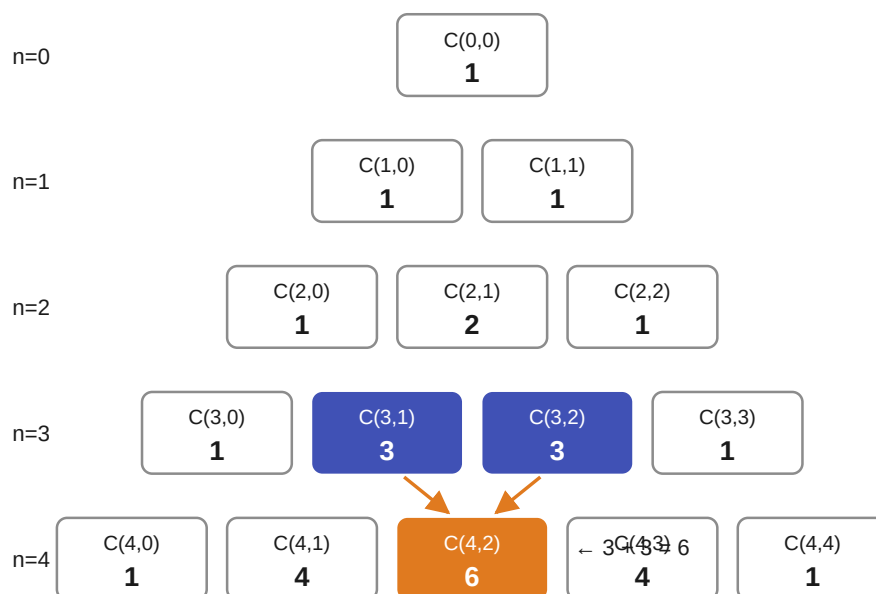
- quelli che **contengono** x : ne restano da scegliere $k - 1$ tra gli altri $n - 1$, quindi sono $\binom{n-1}{k-1}$;
- quelli che **non** contengono x : tutti i k elementi vanno scelti tra gli altri $n - 1$, quindi sono $\binom{n-1}{k}$.

Sommando le due famiglie si ottengono tutti i sottoinsiemi:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

Nell'esempio era $n = 4$, $k = 2$, $x = A$. La formula dice che un coefficiente della riga n si ottiene sommando due coefficienti della riga $n - 1$, la riga precedente. Mettiamo i valori in fila, riga per riga, e la regola diventa visibile.

Scrivendo i valori riga per riga, con $\binom{n}{0} = \binom{n}{n} = 1$ ai bordi, si ottiene il **triangolo di Pascal**. In ogni casella sono scritti il coefficiente binomiale (nella forma $C(n,k)$, cioè « n su k ») e il suo valore. La regola dice che ogni valore è la somma dei due che stanno sopra di lui, a sinistra e a destra. Nella figura è evidenziato il caso dell'esempio dei server: $\binom{4}{2} = \binom{3}{1} + \binom{3}{2} = 3 + 3 = 6$.



Il triangolo di Pascal fino a $n = 4$. Ogni casella contiene il coefficiente binomiale $C(n,k)$ (« n su k ») e il suo valore. Le due caselle blu, sopra a sinistra e a destra, si sommano nella casella arancione sotto di loro. Ai bordi c'è sempre 1.

Continuando con la stessa regola si ottengono le righe successive: 1, 5, 10, 10, 5, 1 per $n = 5$, e così via.

Pascal vi dedicò un trattato scritto in quegli anni, pubblicato dopo la sua morte; il triangolo era però già noto in altre tradizioni matematiche, e il nome è una convenzione. Ciò che conta per noi è che questi numeri li abbiamo già incontrati.

Ricordiamo il Problema dei punti. Due giocatori, A e B , giocano una partita a manche; la partita si interrompe e bisogna dividere la posta in proporzione alle probabilità di vittoria. Con il metodo di Fermat si fanno giocare comunque $n = a + b - 1$ manche, dove a e b sono i punti che mancano ad A e a B . Ognuno dei 2^n futuri possibili, scritto come parola di n lettere A e B , ha la stessa probabilità $\frac{1}{2^n}$, e A vince la partita quando in quel futuro vince **almeno a manche**.

Nel secondo caso dell'Atto II, il gioco a 3 punti sull'1-0, era $a = 2$, $b = 3$, $n = 4$. Avevamo raggruppato i 16 futuri per numero k di manche vinte da A e li avevamo contati a mano: 1, 4, 6, 4, 1 per $k = 0, 1, 2, 3, 4$. Sono i numeri della riga $n = 4$ del triangolo, e non per caso. Un futuro in cui A vince esattamente k manche si può descrivere dicendo **quali** sono le k manche vinte da A , tra le n in programma, e le altre le vince B . Per esempio $BABA$ è il futuro in cui A vince la 2^a e la 4^a manche. Scegliere un futuro con k vittorie di A equivale quindi a scegliere un sottoinsieme di k posizioni tra n , e di sottoinsiemi così ce ne sono $\binom{n}{k}$. Con $n = 4$ e $k = 2$ si ottiene $\binom{4}{2} = 6$, esattamente le 6 parole della riga $k = 2$ della tabella.

Questo ci permette di scrivere la probabilità di Fermat senza più elencare i futuri. Si sommano i $\binom{n}{k}$ per tutti i k che fanno vincere A , cioè da $k = a$ fino a $k = n$, e si divide per il numero totale di futuri:

$$P(A \text{ vince}) = \frac{1}{2^n} \sum_{k=a}^n \binom{n}{k}.$$

Controllo sul caso di prima ($a = 2, n = 4$): $\frac{1}{16} \left(\binom{4}{2} + \binom{4}{3} + \binom{4}{4} \right) = \frac{6+4+1}{16} = \frac{11}{16}$. Sul gioco a 5 punti ($a = 3, n = 7$) lo stesso calcolo dà $\frac{35+35+21+7+1}{128} = \frac{99}{128}$, senza scrivere i 128 futuri: ecco il conteggio che nell'Atto II avevamo rimandato a questo punto.

Si noti che, sommando tutti i numeri di una riga, si ottengono *tutti* i futuri possibili: $\sum_k \binom{n}{k} = 2^n$. Si noti anche che 2^n coincide con il numero di sottoinsiemi di un insieme di n elementi (cioè la cardinalità dell'*insieme delle parti*).

Perché i sottoinsiemi sono 2^n . Per contare tutti i sottoinsiemi di un insieme di n elementi, vediamo in quanti modi se ne può *costruire* uno. Prendiamo gli elementi uno alla volta. Il primo può essere incluso o escluso dal sottoinsieme: due possibilità. Anche il secondo può essere incluso o escluso, e questa scelta non dipende dalla precedente, quindi le possibilità si **raddoppiano**. Lo stesso vale per il terzo, il quarto, fino all' n -esimo. Le scelte possibili sono quindi

$$\overbrace{2 \cdot 2 \cdot 2 \cdots 2}^{n \text{ volte}} = 2^n.$$

Con $n = 3$ si ha $2 \cdot 2 \cdot 2 = 8$, e infatti i sottoinsiemi di $\{a, b, c\}$ sono otto: $\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}$. Aggiungere un solo elemento raddoppia il numero di sottoinsiemi: gli 8 di prima restano tali, e se ne aggiungono altri 8 uguali con il nuovo elemento dentro.

Lo stesso conteggio, fatto raggruppando i sottoinsiemi per numero k di elementi (ce ne sono $\binom{n}{k}$ per ogni k), dà $\sum_k \binom{n}{k}$. Poiché i due modi di contare riguardano gli stessi oggetti, i risultati coincidono: è la formula $\sum_k \binom{n}{k} = 2^n$ scritta sopra.

La probabilità classica, e il suo limite nascosto

Ora possiamo enunciare con precisione l'idea di Laplace della Settimana 1: se Ω è finito e **tutti i suoi esiti sono equiprobabili**,

$$P(A) = \frac{|A|}{|\Omega|} = \frac{\text{casi favorevoli}}{\text{casi possibili}}.$$

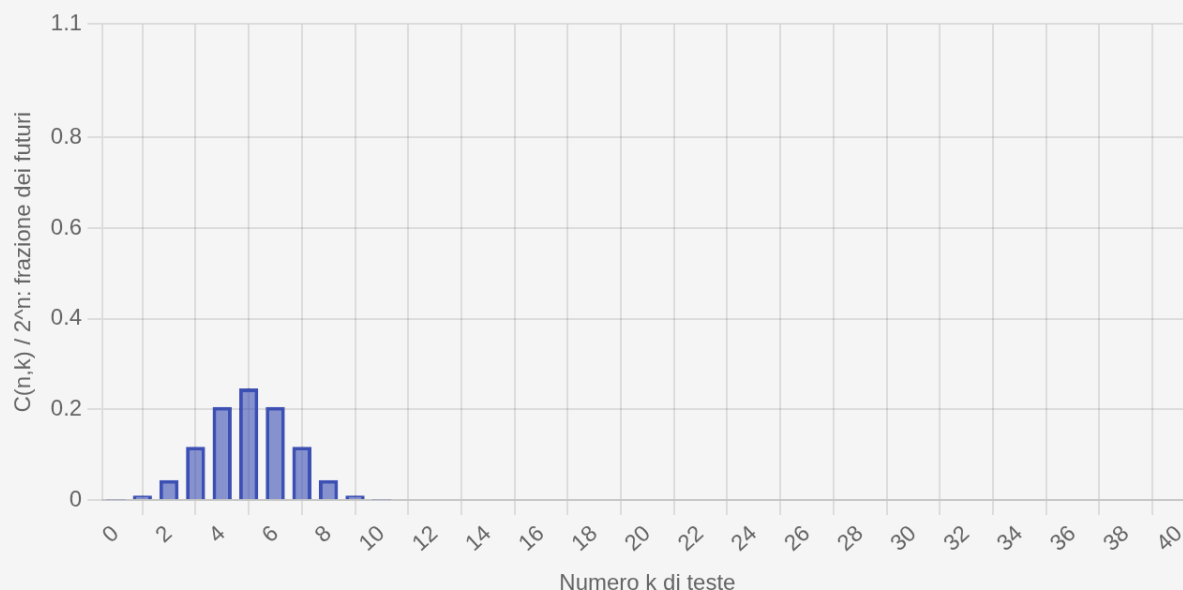
Tutto il lavoro sta nel contare numeratore e denominatore con gli strumenti di sopra. Per esempio, la probabilità di ottenere esattamente 5 teste in 10 lanci di una moneta onesta è $\binom{10}{5}/2^{10} = 252/1024 \approx 0,246$.

L'ipotesi «equiprobabili» però non è un dettaglio, e sbagliare **che cosa** contare è l'errore classico, e ci siamo già cascati all'inizio della settimana con le due monete di d'Alembert: contare tre esiti (T, CT, CC) invece delle quattro sequenze ordinate, tutte di probabilità $\frac{1}{4}$. Anche se si ragiona in termini di «numero di teste», gli esiti possibili sono tre (0, 1, 2) ma **non** sono equiprobabili: «una testa» raccoglie due sequenze, TC e CT . Lo stesso succede con la somma di due dadi: le 11 somme possibili (2, ..., 12) non sono equiprobabili, mentre le 36 coppie ordinate sì. La regola pratica: **per usare «casi favorevoli su casi possibili» bisogna contare oggetti che si presentino davvero con la stessa probabilità**, e per questo conviene quasi sempre distinguere gli esiti (due monete *distinguibili*, due dadi *distinguibili*).

Questa regola avrà un volto concreto tra due sezioni: una funzione hash ben progettata è, per definizione, un meccanismo che rende equiprobabili gli slot in cui finiscono le chiavi.

Esplora tu stesso: quanti futuri hanno k vittorie?

Il grafico mostra, per n lanci di una moneta onesta, la frazione $\binom{n}{k}/2^n$ dei futuri con esattamente k teste. Aumenta n : i valori centrali dominano sempre di più.



Con $n = 4$ si legge la riga 1, 4, 6, 4, 1 del triangolo di Pascal, divisa per 16.

Futuri totali 2^n : **1024** Più frequente: $k = 5$: **24.6%**

Script 2 – Le formule contro l'enumerazione

Prima di fidarci delle formule le confrontiamo con l'enumerazione esplicita su casi piccoli, costruiamo il triangolo di Pascal con la regola della somma ($\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$), e verifichiamo con una simulazione la probabilità classica dei 5 teste su 10 lanci.

Blocco di codice di 48 righe omissso dalla versione PDF: lo trovi, e puoi eseguirlo direttamente nel browser, nella versione online del capitolo.

Atteso: 125 sequenze, 60 disposizioni e 10 combinazioni; il triangolo di Pascal stampato fino alla riga 6 (1, 6, 15, 20, 15, 6, 1); 0.2461 esatto e circa 0.246 simulato; $\log_2(100!) \approx 524,8$.

🔗 Come si legge la riga `len(list(product(range(n), repeat=k)))`, un pezzo alla volta

La riga sembra fitta, ma sono quattro idee semplici, una dentro l'altra. Leggiamola **da dentro verso fuori**, con $n = 5$ e $k = 3$.

1. **`range(n)` : le scelte possibili per un posto.** `range(5)` produce i numeri 0, 1, 2, 3, 4. Pensali come le 5 chiavi (o i 5 slot) tra cui scegliere.
2. **`product(range(n), repeat=k)` : tutte le sequenze di lunghezza k .** `repeat=3` significa «ripeti questa scelta per 3 posti». È come scrivere `product(range(5), range(5), range(5))` : per il primo posto si pesca da 0..4, per il secondo di nuovo da 0..4, per il terzo ancora da 0..4, **ogni volta liberamente**, anche ripetendo lo stesso valore. I risultati sono terne ordinate:

```
(0, 0, 0)
(0, 0, 1)
(0, 0, 2)
...
(4, 4, 3)
(4, 4, 4)
```

Si procede come un contachilometri: l'ultimo posto cambia più in fretta, poi scatta quello prima, e così via.

3. **`list(...)` : metti tutte le terne in una lista.** `product` non le costruisce subito tutte: le produce una alla volta, quando qualcuno le chiede. `list(...)` le chiede **tutte** e le raccoglie in una lista.

4. **`len(...)` : quante sono.** `len` conta gli elementi della lista. Il risultato è 125.

Perché 125? Per la regola della moltiplicazione: 5 scelte per il primo posto, 5 per il secondo, 5 per il terzo, quindi $5 \cdot 5 \cdot 5 = 5^3 = 125$. L'`assert` controlla che **contare a mano** con la formula `n**k` e **elencare davvero** tutte le terne diano lo stesso numero.

Per riprodurre l'idea in piccolo, con $n = 2$ e $k = 2$: `list(product(range(2), repeat=2))` dà `[(0, 0), (0, 1), (1, 0), (1, 1)]`, cioè le $2^2 = 4$ sequenze ordinate di due monete, le stesse quattro di d'Alembert.

Atto IV: Compleanni e coincidenze — il trucco del complementare

La risposta a de Méré

Torniamo allo «scandalo» dell'Atto I e chiediamoci: qual è la probabilità di ottenere **almeno un** sei lanciando un dado quattro volte?

Quante sono le sequenze possibili? Un lancio ha 6 esiti. Con due lanci, per ciascuno dei 6 esiti del primo ce ne sono 6 del secondo: sono $6 \cdot 6 = 36$ coppie, come nella tabella della Settimana 1. Ogni lancio in più **moltiplica per 6** il numero di sequenze (è lo stesso ragionamento dell'albero della Settimana 2: ogni foglia si ramifica in 6). Con quattro lanci:

$$6 \cdot 6 \cdot 6 \cdot 6 = 1296 \text{ sequenze, tutte equiprobabili.}$$

(Si scrive in breve 6^4 : «6 moltiplicato per sé stesso 4 volte».)

Quante hanno almeno un sei? Contarle direttamente è lungo: un sei, due sei, tre, quattro, e per ognuno *in quali lanci*. Conviene contare il caso opposto, **nessun sei**, e usare il complementare (Settimana 1, §6B). Se non deve uscire il 6, a ogni lancio restano 5 esiti (1, 2, 3, 4, 5), quindi

$$5 \cdot 5 \cdot 5 \cdot 5 = 625 \text{ sequenze senza sei.}$$

Le altre hanno almeno un sei: $1296 - 625 = 671$. Sono i «671 contro 625» di Pascal: **671 sequenze vincenti contro 625 perdenti**. La probabilità è

$$P(\text{almeno un sei in 4 lanci}) = \frac{671}{1296} = 1 - \left(\frac{5}{6}\right)^4 \approx 0,518.$$

Con due dadi. Un lancio ha 36 coppie, e il doppio sei è **una sola** di esse: la coppia (6, 6). Le altre 35 *non* sono un doppio sei, comprese quelle con un solo 6 come (6, 1) o (2, 6). In 24 lanci le sequenze possibili sono 36^{24} (36 moltiplicato per sé stesso 24 volte) e quelle senza alcun doppio sei sono 35^{24} . I numeri sono enormi, ma conta il rapporto:

$$P(\text{almeno un doppio sei in 24 lanci}) = 1 - \frac{35^{24}}{36^{24}} = 1 - \left(\frac{35}{36}\right)^{24} \approx 0,491.$$

Quindi non c'è alcuna contraddizione: la prima scommessa è favorevole (51,8%), la seconda no (49,1%).

Dov'era l'errore di de Méré. Ragionava per proporzione: 4 lanci su 6 facce, 24 lanci su 36 coppie, stesso rapporto $\frac{2}{3}$. Equivale a «sommare» le probabilità: $4 \cdot \frac{1}{6} = \frac{2}{3}$ e $24 \cdot \frac{1}{36} = \frac{2}{3}$. Ma sommare funziona solo per eventi **disgiunti** (Settimana 1, §6C), mentre «sei al primo lancio» e «sei al secondo» possono verificarsi insieme, e quelle sequenze verrebbero contate più volte.

Quanti lanci servono davvero? La scommessa conviene quando le sequenze senza successo sono **meno della metà** del totale.

lanci n	un dado: $2 \cdot 5^n$ contro 6^n	conviene?	due dadi: $2 \cdot 35^n$ contro 36^n	conviene?
3	$250 > 216$	no (42,1%)		
4	$1250 < 1296$	sì (51,8%)		
24			$2 \cdot 35^{24} > 36^{24}$	no (49,1%)
25			$2 \cdot 35^{25} < 36^{25}$	sì (50,6%)

Con un dado il minimo è 4 lanci, e de Méré lo sapeva per esperienza. Con due dadi la proporzione dà $4 \cdot 6 = 24$, ma il numero giusto è 25: un lancio in più, ed è proprio quel lancio che lui non vedeva.

**Il trucco del complementare**

Quando la domanda è «**almeno uno...**», calcola la probabilità di «**nessuno...**» (di solito un prodotto) e sottrai da 1. Lo useremo di nuovo adesso, e ancora per le tabelle hash.

Script 3 – De Méré, esatto e simulato

Blocco di codice di 49 righe omissso dalla versione PDF: lo trovi, e puoi eseguirlo direttamente nel browser, nella versione online del capitolo.

Il paradosso dei compleanni

Ecco una domanda che sembra innocua. **Quante persone devono essere presenti in una stanza perché la probabilità che almeno due compiano gli anni lo stesso giorno superi il 50%?** Chiunque risponde con un numero grande (183? 100?). La risposta è 23.

Modelliamo il problema con ipotesi esplicite: n persone, 365 giorni possibili (ignoriamo il 29 febbraio), ogni persona nata in un giorno scelto uniformemente e indipendentemente dalle altre (anche questa è un'assunzione semplificatrice). Lo spazio campionario è l'insieme delle sequenze di n giorni, e **per la moltiplicazione** ha 365^n elementi tutti equiprobabili. Applichiamo il trucco del complementare: invece di contare i casi con *almeno una* coincidenza, contiamo quelli con **tutti i compleanni diversi**, che sono più facili da descrivere.

Un esempio con $n = 3$ persone. Assegniamo i compleanni uno alla volta:

- la prima persona può avere qualunque giorno: **365** scelte;
- la seconda deve evitare il giorno della prima: **364** scelte;
- la terza deve evitare i due giorni già occupati: **363** scelte.

Per la regola della moltiplicazione le sequenze con tutti i giorni diversi sono $365 \cdot 364 \cdot 363$, su un totale di 365^3 sequenze possibili. Quindi

$$P(\text{nessuna coincidenza}) = \frac{365 \cdot 364 \cdot 363}{365^3}.$$

Con n persone il ragionamento è identico: al numeratore ci sono n fattori che scendono di uno alla volta, a partire da 365 (sono le disposizioni semplici dell'Atto III); al denominatore c'è 365^n :

$$P(\text{nessuna coincidenza}) = \frac{365 \cdot 364 \cdots (365 - n + 1)}{365^n}.$$

Una forma più comoda. Il denominatore 365^n è 365 moltiplicato per sé stesso n volte: possiamo assegnare un 365 a ciascun fattore del numeratore.

$$\frac{365}{365} \cdot \frac{364}{365} \cdot \frac{363}{365} \cdots \frac{365 - n + 1}{365}$$

Ogni frazione si riscrive come $\frac{365-i}{365} = 1 - \frac{i}{365}$, dove i vale $0, 1, 2, \dots, n-1$. Il simbolo $\prod$ è solo la scrittura compatta di «moltiplica tutti questi fattori», come $\sum$ lo è per la somma:

$$P(\text{nessuna coincidenza}) = \prod_{i=0}^{n-1} \left(1 - \frac{i}{365}\right).$$

Per $n = 3$ i fattori sono $1 \cdot \left(1 - \frac{1}{365}\right) \cdot \left(1 - \frac{2}{365}\right) = \frac{365}{365} \cdot \frac{364}{365} \cdot \frac{363}{365}$, cioè la stessa espressione di prima.

Questo prodotto si legge anche con la regola del prodotto: la seconda persona evita il compleanno della prima con probabilità $\frac{364}{365}$, la terza evita i due già presi con probabilità $\frac{363}{365}$, e così via. La probabilità di **almeno una coincidenza** è $1 - \prod_{i=0}^{n-1} \left(1 - i/365\right)$:

n	10	20	23	30	40	50
$P(\text{coincidenza})$	11,7%	41,1%	50,7%	70,6%	89,1%	97,0%

Perché è così sorprendente?

La nostra intuizione confronta **la mia data** con quelle degli altri: 22 persone hanno circa il 6% di probabilità di condividere *il mio* compleanno ($1 - (364/365)^{22} \approx 5,9\%$). Ma il paradosso chiede se **qualunque coppia** coincide, e le coppie sono molte di più delle persone: in una stanza di n persone ci sono $\binom{n}{2} = \frac{n(n-1)}{2}$ coppie, cioè **253** per $n = 23$. Ognuna di queste coppie ha una probabilità $\frac{1}{365}$ di coincidere, e ogni coppia è un'occasione in più per trovare una coincidenza. Il numero di coppie cresce come n^2 , non come n : è questo che l'intuizione sottovaluta.

L'approssimazione esponenziale e la regola della radice

La formula esatta è un prodotto di n fattori: per $n = 23$ sono 23 moltiplicazioni, e non si vede a colpo d'occhio come il risultato dipenda da n . Vogliamo una formula corta, anche a costo di essere un po' approssimata. Lavoriamo con N al posto di 365, così il ragionamento vale per qualunque numero di «giorni».

Passo 1: un fatto sull'esponenziale. Per ogni x vale

$$1 - x \leq e^{-x},$$

e le due quantità sono molto vicine quando x è piccolo. Un controllo con qualche numero:

x	0,01	0,1	0,5
$1 - x$	0,99	0,9	0,5
e^{-x}	0,9900	0,9048	0,6065

Per $x = 0,01$ la differenza si vede alla quarta cifra decimale; per $x = 0,5$ no. Geometricamente, $y = 1 - x$ è la retta tangente alla curva $y = e^{-x}$ nel punto $x = 0$, e la curva sta sempre sopra la sua tangente.

Passo 2: applichiamo a ogni fattore. I fattori del prodotto hanno la forma $1 - x$ con $x = \frac{i}{N}$, che è piccolo finché i è piccolo rispetto a N . Sostituiamo ciascun fattore con il suo $e^{-i/N}$, che è un po' più grande (e anche positivo, quindi moltiplicare le disuguaglianze è lecito):

$$\prod_{i=0}^{n-1} \left(1 - \frac{i}{N}\right) \leq \prod_{i=0}^{n-1} e^{-i/N}.$$

Passo 3: trasformiamo il prodotto in una somma. L'esponenziale ha la proprietà $e^a \cdot e^b = e^{a+b}$: moltiplicare esponenziali equivale a sommare gli esponenti. Quindi

$$\prod_{i=0}^{n-1} e^{-i/N} = e^{-\frac{0}{N}} \cdot e^{-\frac{1}{N}} \cdots e^{-\frac{n-1}{N}} = e^{-\frac{1}{N} (0+1+2+\cdots+(n-1))}.$$

Passo 4: sommiamo gli interi. La somma $0 + 1 + \cdots + (n-1)$ si calcola come faceva Gauss: si scrive due volte, una in avanti e una all'indietro, e si sommano le colonne.

$$\begin{array}{ccccccc} 0 & + & 1 & + & \cdots & + & (n-1) \\ (n-1) & + & (n-2) & + & \cdots & + & 0 \end{array}$$

Ogni colonna somma $n-1$, e le colonne sono n . Il doppio della somma è quindi $n(n-1)$, e la somma vale $\frac{n(n-1)}{2}$. In conclusione

$$\prod_{i=0}^{n-1} \left(1 - \frac{i}{N}\right) \leq e^{-\frac{n(n-1)}{2N}}.$$

Passo 5: passiamo alla probabilità di coincidenza. Il prodotto è la probabilità di *nessuna* coincidenza, e la probabilità cercata è 1 meno questa. Sottraendo da 1 la disuguaglianza si **capovolge**:

$$P(\text{almeno una coincidenza}) \geq 1 - e^{-\frac{n(n-1)}{2N}} \approx 1 - e^{-\frac{n^2}{2N}}.$$

L'ultimo passaggio sostituisce $n(n-1)$ con n^2 , che è ragionevole quando n è grande. Siccome la stima è un minimo, è una stima **per difetto**: la probabilità vera è almeno questa. È accurata finché n è molto più piccolo di N (perché serve che gli $x = i/N$ siano piccoli).

Controllo con $N = 365$ e $n = 23$. Qui $n(n-1) = 23 \cdot 22 = 506$, quindi l'esponente è $\frac{506}{2 \cdot 365} = \frac{506}{730} \approx 0,69$. La stima vale $1 - e^{-0,69} \approx 0,500$, contro il valore vero 0,507: sbaglia di meno di un punto percentuale.

La regola della radice. Chiediamoci a che n la stima raggiunge $\frac{1}{2}$:

$$1 - e^{-\frac{n^2}{2N}} = \frac{1}{2} \iff e^{-\frac{n^2}{2N}} = \frac{1}{2} \iff \frac{n^2}{2N} = \ln 2 \iff n = \sqrt{2 \ln 2 \cdot N} \approx 1,18 \sqrt{N}.$$

(Passiamo da $e^{-y} = \frac{1}{2}$ a $y = \ln 2$ prendendo il logaritmo naturale di entrambi i lati.) Per $N = 365$ otteniamo $1,18 \cdot 19,1 \approx 22,5$, cioè 23.

La conseguenza è generale: **con N possibilità equiprobabili, bastano circa $\sqrt{N}$ estrazioni perché una ripetizione diventi probabile**, non N . Per 365 giorni è 23, per un milione di «giorni» sono circa 1180.

Esplora tu stesso: una sola curva per tutti gli N

Finora abbiamo ragionato su $N = 365$. Ma la regola della radice dice che il risultato vale per *qualsiasi* N : cambia solo la scala. Il seguente grafico lo fa vedere.

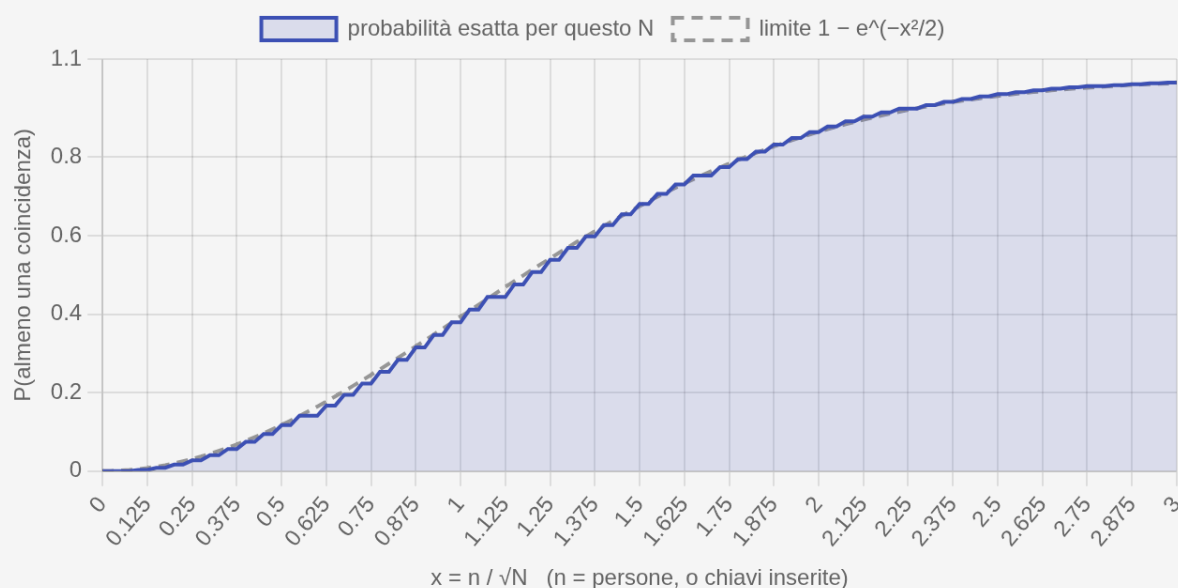
- **Asse orizzontale:** invece del numero di persone n , usiamo $x = n/\sqrt{N}$, cioè contiamo le persone **in unità di $\sqrt{N}$** . Per $N = 365$ si ha $\sqrt{N} \approx 19$, quindi $n = 23$ persone corrispondono a $x \approx 1,2$; con $N = 1\,000\,000$ si ha $\sqrt{N} = 1000$, e $x = 1,2$ corrisponde a $n = 1200$ persone.
- **Asse verticale:** la probabilità *esatta* (calcolata con il prodotto del Passo 1, senza approssimazioni) che ci sia almeno una coincidenza.
- **Cursore:** sceglie N , da 10 a 10 milioni (scala logaritmica: ogni passo moltiplica N per un fattore costante).

Cosa osservare. Sposta il cursore da un N piccolo a uno enorme: la curva **resta quasi identica**. Significa che, se misuriamo le persone in unità di $\sqrt{N}$, la probabilità dipende solo da x e non dal valore di N .

Questo è previsto dalla formula approssimata. Se nella stima $1 - e^{-n^2/(2N)}$ scriviamo $n = x\sqrt{N}$, otteniamo $n^2 = x^2N$, e il N al numeratore si semplifica con quello al denominatore:

$$1 - e^{-\frac{x^2N}{2N}} = 1 - e^{-x^2/2}.$$

Il N è sparito: ecco perché una sola curva (la linea tratteggiata) va bene per tutti. Infine, la curva passa per $\frac{1}{2}$ quando $x \approx 1,18$, cioè $n \approx 1,18\sqrt{N}$: è la regola della radice trovata sopra.



La linea tratteggiata è la curva per $N = 10$ milioni, praticamente il limite. Con $N = 365$ la curva esatta è appena sotto.

N: 365 Prime n con $P \geq 50\%$: **23** Regola $1,18 \cdot \sqrt{N}$: **22.5**

Script 4 – Compleanni: esatto, approssimato, simulato

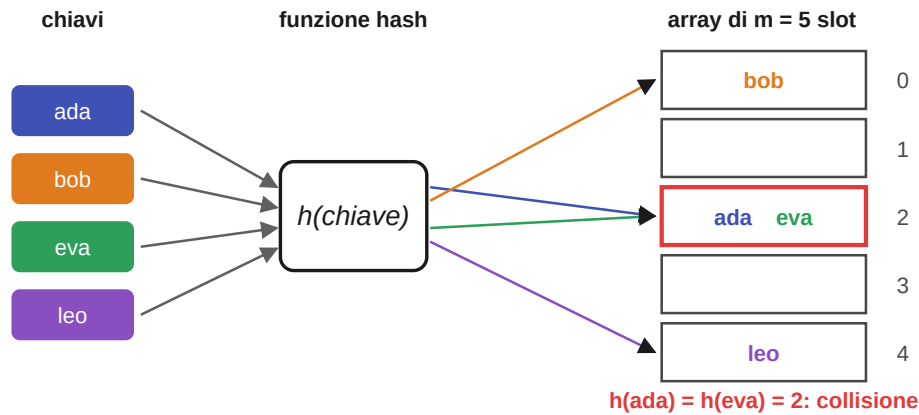
Blocco di codice di 53 righe omesso dalla versione PDF: lo trovi, e puoi eseguirlo direttamente nel browser, nella versione online del capitolo.

Atteso: la tabella con 0.5073 per $n = 23$; soglie 23, 119, 1178 per $N = 365, 10^4, 10^6$; e 0.0586 nell'ultima riga.

Atto V: Tabelle hash e Birthday Attack

Una tabella hash è un tavolo da compleanni

Una **tabella hash** è un array di m posizioni (*slot*). Per inserire una chiave si applica una **funzione hash** h che la trasforma in un numero tra 0 e $m - 1$: la chiave va nello slot $h(\text{chiave})$. Se due chiavi diverse finiscono nello stesso slot si ha una **collisione**, e la struttura dati deve prevedere come gestirla (liste concatenate, indirizzamento aperto...).



La funzione hash h trasforma ogni chiave in un indice tra 0 e $m-1$. Qui $h(bob) = 0$ e $h(leo) = 4$, mentre «ada» ed «eva» finiscono nello stesso slot.

Una funzione hash ben progettata «mescola» le chiavi come se ogni chiave scegliesse il proprio slot **a caso, uniformemente e indipendentemente dalle altre**: nei testi di algoritmi è l'ipotesi di *simple uniform hashing* (per esempio nel cap. 11 di Cormen et al., *Introduction to Algorithms*). È un modello, non un fatto: una funzione hash reale approssima questo comportamento, e quando non lo fa le collisioni peggiorano. Ma sotto questa ipotesi il problema è **identico** al paradosso dei compleanni: gli slot sono i giorni, le chiavi sono le persone, $N = m$.

Quindi, per n chiavi in m slot:

$$P(\text{almeno una collisione}) = 1 - \prod_{i=0}^{n-1} \left(1 - \frac{i}{m}\right) \geq 1 - e^{-\frac{n(n-1)}{2m}},$$

e la collisione diventa **probabile** (oltre il 50%) quando $n \approx 1,18\sqrt{m}$. Con una tabella da **un milione di slot**, bastano circa **1180 chiavi**, cioè lo 0,12% della capienza, perché due chiavi condividano uno slot. Con 1000 chiavi la probabilità di avere almeno una collisione è già vicina al 40%. Morale della favola: una tabella quasi vuota *non* è una tabella senza collisioni.

In ogni caso, questo fatto è una buona notizia travestita da cattiva: le collisioni non sono un difetto da eliminare, ma una certezza da gestire. Chi si aspetta che «con una buona funzione hash non succeda mai» sbaglia il modello; chi le prevede dimensiona le liste e il fattore di carico di conseguenza.

✎ Perché è una buona notizia: collisioni frequenti, ma innocue



Il paradosso dei compleanni ci dice che una collisione compare quasi subito. Ma a noi non interessa la collisione in sé: interessa se la tabella **continua a fare il suo lavoro bene**, cioè se cercare e inserire una chiave resta veloce. Qui «veloce» e «lenta» hanno un significato preciso, che si misura contando i **confronti tra chiavi**:

- Se la chiave cercata sta da sola nel suo slot, basta calcolare h e fare un confronto. Sono pochi passi, sempre gli stessi, qualunque sia il numero n di chiavi memorizzate.
- Se nello slot ci sono altre k chiavi finite lì per collisione, bisogna scorrerle e confrontarle una per una: circa k passi.
- Nel caso peggiore, tutte le n chiavi finiscono nello stesso slot. La ricerca diventa una scansione di tutti gli elementi, come in una semplice lista: $O(n)$. Questa è una tabella davvero «lenta».

Quindi «c'è almeno una collisione» e «la tabella è lenta» sono affermazioni molto diverse. La prima riguarda l'**esistenza** delle collisioni, e il paradosso dei compleanni risponde: quasi subito sì. La seconda riguarda la **lunghezza** delle liste che si formano, perché è quella a determinare quanti confronti servono. Una collisione isolata, che aggiunge un confronto a una ricerca ogni tanto, è innocua; sono le liste lunghe a rovinare le prestazioni. Per capire quanto lunghe diventano, serve una grandezza che misuri quanto è piena la tabella.

Si definisce **fattore di carico** il rapporto

$$\alpha = \frac{n}{m},$$

cioè il numero medio di chiavi per slot. Con la risoluzione per **liste concatenate** (ogni slot contiene una lista delle chiavi che vi sono finite), sotto l'ipotesi di *simple uniform hashing* la lunghezza attesa di una lista è esattamente α . Cercare una chiave costa quindi, in media, un calcolo di h più la scansione di una lista di lunghezza media α : un costo atteso $\Theta(1 + \alpha)$ (Cormen et al., cap. 11). **Non dipende da n** , ma solo dal rapporto tra n ed m .

Rimettiamo insieme i numeri dell'esempio:

- $m = 10^6$ e $n = 1000$ chiavi: $\alpha = 0,001$. La collisione c'è con probabilità ~40%, ma il numero atteso di coppie in collisione è $\binom{n}{2}/m \approx 0,5$: **una mezza collisione in tutta la tabella**, e una ricerca tocca quasi sempre liste vuote o di un solo elemento.
- $n = m$, cioè la tabella *piena* ($\alpha = 1$): la frazione di slot che resta vuota è circa $1/e \approx 37\%$ e ci sono slot con più chiavi, ma la lista più lunga è di poche unità (cresce come $\ln n / \ln \ln n$, per un milione di chiavi dell'ordine di 8–10). Una ricerca costa in media circa una o due letture.

In altre parole: le collisioni **sono molte, ma corte**. Il loro costo si paga solo come piccola costante, ed è per questo che una tabella hash resta $O(1)$ in media anche se «in teoria» continua a collidere.

E se non si usano le liste? Con l'**indirizzamento aperto** (le chiavi stanno nell'array stesso e, in caso di collisione, si prova lo slot successivo secondo una regola fissata) il discorso è lo stesso ma più severo: serve $\alpha < 1$ e il costo atteso di una ricerca senza successo cresce come $\frac{1}{1-\alpha}$ (sotto l'ipotesi di *uniform hashing*). A $\alpha = 0,5$ sono circa 2 tentativi, a $\alpha = 0,9$ circa 10: la tabella degrada rapidamente quando si riempie.

Cosa fanno le librerie, in pratica? Tengono α sotto una soglia e, quando la superano, **ridimensionano** (*rehashing*): allocano una tabella più grande, di solito il doppio, e reinseriscono tutte le chiavi. Il costo del rehash è ammortizzato sulle inserzioni, quindi l'inserimento resta $O(1)$ ammortizzato. Alcuni esempi:

- `HashMap` di Java: liste concatenate (trasformate in alberi bilanciati se una lista diventa troppo lunga) e ridimensionamento a $\alpha = 0,75$ per default;
- `dict` di CPython: indirizzamento aperto e ridimensionamento intorno a $\alpha = 2/3$.

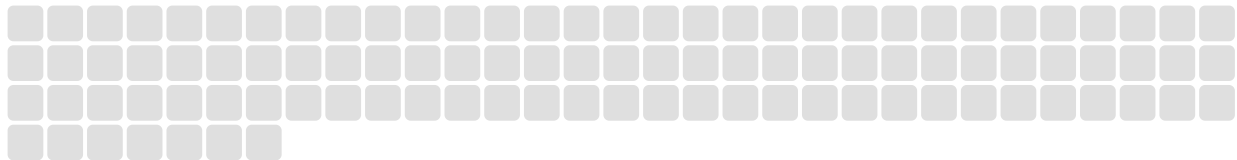
Le soglie sono scelte di implementazione: non vanno memorizzate, ma controllate nella documentazione della versione che si usa.

La morale operativa. Il progettista di una tabella hash non chiede alla funzione hash di *evitare* le collisioni (impossibile: il paradosso dei compleanni dice che sono quasi immediate) ma di *distribuire* le chiavi in modo che nessuno slot si carichi troppo, e poi governa α con il ridimensionamento. Il rischio vero compare quando la distribuzione **non** è uniforme, per esempio perché qualcuno **sceglie le chiavi apposta** per farle cadere nello stesso slot: in quel caso le liste diventano lunghe e le operazioni degenerano verso $O(n)$ (è la base degli attacchi di *hash flooding* contro i server web, che è il motivo per cui molti linguaggi usano funzioni hash con seme casuale). Nella sezione successiva vediamo l'altra faccia della medaglia: quando a cercare le collisioni è un attaccante e non il caso, lo stesso calcolo dei compleanni diventa un'arma.

Esplora tu stesso: riempi la tabella

Scegli il numero di slot m e inserisci chiavi casuali, una alla volta o dieci alla volta. Gli slot vuoti sono grigi, quelli occupati da una chiave sono blu, quelli con una collisione sono arancioni. Dopo quante chiavi compare la prima collisione? Ripeti l'esperimento più volte: confronta con $\sqrt{m}$.

Slot m:  **100**
 +1 chiave +10 chiavi Azzera



Chiavi inserite: **0** · slot occupati: **0** · chiavi finite in uno slot già occupato: **0**
 Prima collisione: **non ancora** · regola $1,18 \cdot \sqrt{m} \approx 11,8$
 P(almeno una collisione con 0 chiavi) = **0.0%**

Il Birthday Attack

Una funzione hash crittografica (SHA-256, per esempio) trasforma qualunque messaggio in un valore di b bit, quindi $N = 2^b$ possibili «giorni». Una proprietà richiesta è la **resistenza alle collisioni**: deve essere difficile trovare **due** messaggi diversi con lo stesso hash. L'attaccante non ha bisogno di colpire un valore prefissato (che costerebbe in media circa 2^b tentativi): gli basta trovare una qualunque coppia che coincide, e per la regola della radice servono circa $1,18 \cdot 2^{b/2}$ messaggi. Questo è il **birthday attack**.

Bit dell'hash b	32	64	128	160	256
Messaggi per il 50% di collisione ($\approx 1,18 \cdot 2^{b/2}$)	$7,7 \cdot 10^4$	$5,1 \cdot 10^9$	$2,2 \cdot 10^{19}$	$1,4 \cdot 10^{24}$	$4,0 \cdot 10^{38}$

Si legge subito che un hash di 32 bit si rompe con una manciata di decine di migliaia di messaggi, e che 64 bit sono alla portata di un calcolo serio. Ma la tabella nasconde una conseguenza più importante: **la sicurezza contro le collisioni è la metà dei bit dell'hash**. Vediamo perché, in tre passaggi.

1. **Due attacchi diversi.** Se l'attaccante deve trovare un messaggio con *uno specifico* hash (o con lo stesso hash di un messaggio dato), è come cercare chi compie gli anni *proprio oggi*: servono circa 2^b tentativi. Se invece gli basta trovare *una qualunque coppia* di messaggi con lo stesso hash, è il paradosso dei compleanni: bastano circa $\sqrt{2^b} = 2^{b/2}$ messaggi. La resistenza alle collisioni riguarda il secondo caso, quindi il costo dell'attacco è $2^{b/2}$ e non 2^b .
2. **Misurare la sicurezza in bit.** Si dice che un sistema ha «sicurezza a k bit» quando rompere richiede circa 2^k operazioni. Un attacco da $2^{b/2}$ operazioni dà quindi $k = b/2$.
3. **Il risultato.** SHA-256 produce $b = 256$ bit, ma contro le collisioni offre «solo» $k = 128$ bit di sicurezza: servono circa 2^{128} messaggi (la colonna 256 della tabella, $4,0 \cdot 10^{38}$). Sono comunque un numero fuori portata per qualunque computer.

Per questo i progettisti ragionano al contrario: se si vuole una sicurezza a 128 bit contro le collisioni, l'hash deve avere **il doppio dei bit**, cioè 256.

Una precisazione di realtà: il birthday attack è il limite generale che vale per **qualunque** funzione hash, ma le funzioni difettose cedono molto prima. Per MD5 le prime collisioni pratiche sono state ottenute con attacchi crittanalitici nel 2004 (Wang e Yu, *How to Break MD5 and Other Hash Functions*, Eurocrypt 2005), e per SHA-1 nel 2017 il progetto *SHAttered* ha prodotto due PDF diversi con lo stesso hash, con circa nove miliardi di miliardi di calcoli SHA-1 (circa 2^{63} , contro i circa 2^{80} del birthday attack puro).

Script 5 – Il birthday attack su un hash troncato

Non possiamo rompere SHA-256, ma possiamo verificare la regola della radice su una versione in miniatura: **troncare** l'hash ai suoi primi b bit, con b piccolo (12, 16, 20, 24). Così lo spazio dei valori è $N = 2^b$ e le collisioni arrivano abbastanza presto da poterle osservare.

L'esperimento è lo stesso della tabella hash: si generano messaggi casuali, si calcola l'hash troncato di ciascuno, e ci si ferma alla prima volta che un valore si ripete. Il numero di messaggi hashati fino a quel momento è il risultato di **una prova**.

Una sola prova dipende troppo dalla fortuna (a volte la collisione arriva dopo pochi messaggi, a volte dopo molti di più), quindi lo script ripete la prova molte volte per ogni b e controlla due cose:

1. **La mediana** dei risultati deve essere vicina a $1,18 \cdot 2^{b/2}$: è la regola della radice, cioè il numero di messaggi dopo cui la probabilità di collisione è il 50%. Perciò circa **metà delle prove** deve finire *prima* di questa soglia e *metà dopo*.
2. **Ogni 2 bit in più, i messaggi necessari raddoppiano** (non quadruplicano): lo si vede dalla colonna della mediana, perché cresce come $\sqrt{N}$ e non come N .

Blocco di codice di 44 righe omissa dalla versione PDF: lo trovi, e puoi eseguirlo direttamente nel browser, nella versione online del capitolo.

Atteso: la mediana vicina alla soglia $1,18 \cdot 2^{b/2}$ per ogni b (per esempio circa 300 per $b = 16$ e circa 4800 per $b = 24$), con circa il 50% delle prove sotto la soglia. Passando da $b = 16$ a $b = 24$ ci sono 8 bit in più, cioè 4 raddoppi: la mediana cresce di circa $2^4 = 16$ volte.

Il Takeaway per l'Ingegnere del Software

1. **Le collisioni sono attese, non eccezionali.** Una tabella hash con m slot ha una collisione già dopo circa $\sqrt{m}$ inserimenti. Per questo ogni implementazione seria ha una strategia di risoluzione, e i costi si ragionano in termini di *fattore di carico* n/m , non di «evitare le collisioni».
2. **Con la sicurezza, i bit contano a metà.** Un hash o un identificatore di b bit regge fino a circa $2^{b/2}$ elementi, non 2^b . Un ID di 32 bit scelto a caso è pericoloso già con centomila elementi.
3. **Quando la domanda è «almeno uno», calcola «nessuno».** Il trucco del complementare vale per ogni sistema con molti componenti ciascuno con una piccola probabilità di fallire, in modo indipendente: la probabilità che *almeno uno* fallisca non è la somma delle probabilità dei singoli componenti, perché i casi in cui ne falliscono più d'uno verrebbero contati più volte (la somma è solo un limite superiore). Si può invece sommare le probabilità di «esattamente 1», «esattamente 2», ... fallimenti, che sono eventi disgiunti, ma è più comodo calcolare $1 - \prod_i (1 - p_i)$. La probabilità cresce più in fretta di quanto si pensi. È l'errore di de Méré, trasportato in produzione.
4. **Il numero di coppie cresce come n^2 .** In un sistema con n componenti che possono interferire a due a due, le interazioni possibili sono $\binom{n}{2}$. È questa la ragione per cui i sistemi grandi producono incidenti «improbabili» con regolarità.

Errori da evitare

1. **Confondere «qualcuno coincide con me» con «due qualunque coincidono».** La prima probabilità, con 22 altre persone, è circa 5,9%; la seconda, con 23 persone, è 50,7%. Le coppie sono $\binom{n}{2}$, non n .
2. **Sommare le probabilità di «almeno uno».** Vale solo per eventi disgiunti. Il ragionamento di de Méré dà $\frac{2}{3}$ in due casi che valgono 51,8% e 49,1%.
3. **Dimenticare il $k!$.** Le disposizioni contano l'ordine, le combinazioni no: $D(n, k) = \binom{n}{k} k!$. Se l'ordine non conta e usi $n!/(n-k)!$ conti ogni gruppo $k!$ volte.
4. **Applicare «casi favorevoli su casi possibili» a esiti che non sono equiprobabili.** Tre esiti «0, 1, 2 teste» non sono equiprobabili; le quattro sequenze ordinate sì. Lo stesso vale per le funzioni hash: l'ipotesi di uniformità va giustificata.
5. **Usare l'approssimazione $1 - e^{-n^2/2N}$ quando n non è piccolo rispetto a N .** Vale per $n \ll N$; l'ultima forma n^2 al posto di $n(n-1)$ ignora un termine che conta per n piccolo.
6. **Pensare che «quasi vuota» voglia dire «senza collisioni».** Una tabella da un milione di slot ha il 40% di probabilità di collisione con 1000 chiavi.

Riepilogo

- Il **problema dei punti** (Pacioli 1494, Pascal e Fermat 1654) si risolve pesando i **futuri possibili**: Fermat li conta, Pascal li deduce uno dall'altro con la ricorsione $V(a, b) = \frac{1}{2} V(a-1, b) + \frac{1}{2} V(a, b-1)$.
- Il **conteggio** minimo: moltiplicazione (n^k), disposizioni ($n!/(n-k)!$), combinazioni ($\binom{n}{k}$, ordine ignorato), con il triangolo di Pascal $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ e $\sum_k \binom{n}{k} = 2^n$.
- La **probabilità classica** è $|A|/|\Omega|$ solo se gli esiti contati sono equiprobabili.
- Il **trucco del complementare**: $P(\text{almeno uno}) = 1 - P(\text{nessuno})$. Risolve de Méré e i compleanni: $P = 1 - \prod_{i=0}^{n-1} (1 - i/N) \geq 1 - e^{-n(n-1)/2N}$.
- **Regola della radice**: con N possibilità equiprobabili, la ripetizione è probabile dopo circa $1,18\sqrt{N}$ estrazioni. Tabelle hash: collisioni attese; hash crittografici: la sicurezza contro le collisioni è $b/2$ bit.

Riferimenti e Fonti Primarie (Open Access Online)

Fonti storiche

- **Blaise Pascal e Pierre de Fermat (1654)** — *Corrispondenza sul problema dei punti*, lettere dal 29 luglio al 27 ottobre 1654, ristampate nelle *Œuvres de Fermat*, II, pp. 288–314 (Tannery e Henry, Parigi 1894). Traduzione inglese: pasfer.pdf (sito di Richard J. Pulskamp).
- **Luca Pacioli (1494)** — *Summa de arithmetica, geometria, proportioni et proportionalità*, ff. 197r–198v. Scansione: Internet Archive. Passi di Pacioli e Tartaglia (traduzione inglese): Cantillo, *The Problem of Points*.
- **Niccolò Tartaglia (1556)** — *General trattato di numeri et misure*, parte I, libro XVI, sezione 206 (passo citato in Cantillo, sopra).
- **Christiaan Huygens (1657)** — *De ratiociniis in ludo aleae*. Internet Archive.
- **Jacob Bernoulli (1713)** — *Ars conjectandi*, Basilea. Internet Archive.
- **Il triangolo aritmetico** — storia del triangolo di Pascal e dei suoi predecessori in India, Persia e Cina: Wikipedia, *Pascal's triangle*.
- **Jean Le Rond d'Alembert (1754)** — voce «Croix ou pile», *Encyclopédie*, vol. IV, pp. 512–513 (ottobre 1754); traduzione inglese di R. J. Pulskamp (la nostra traduzione italiana delle citazioni è da questa), e il commento di Publimath.
- **Pierre-Simon Laplace** — *Essai philosophique sur les probabilités*, sezione «Principes généraux du calcul des probabilités», secondo principio: testo su Wikisource.

Storia della probabilità

- **Prakash Gorroochurn** — *Thirteen Correct Solutions to the «Problem of Points» and Their Histories*. PDF dell'autore.

Tabelle hash e sicurezza

- **Cormen, Leiserson, Rivest, Stein** — *Introduction to Algorithms*, 3ª ed.: §5.4.1 («The birthday paradox») e cap. 11 («Hash Tables»).
- **Xiaoyun Wang e Hongbo Yu (2005)** — *How to Break MD5 and Other Hash Functions*, Eurocrypt 2005, LNCS 3494, pp. 19–35.

- **Marc Stevens et al. (2017)** — *The first collision for full SHA-1* (progetto SHAttered): shattered.io.
- **IETF** — *RFC 9562: Universally Unique IDentifiers (UUIDs)*: rfc-editor.org.
- **Wikipedia, Birthday problem** — media del numero di estrazioni e approssimazioni: https://en.wikipedia.org/wiki/Birthday_problem.

Manuale di riferimento e notebook del corso

- **Michael Baron (2014)** — *Probability and Statistics for Computer Scientists*, 2^a ed., CRC Press, §2.3 (combinatoria). Scheda ACM Digital Library.
- **Notebook Colab del corso** (repository colab_handouts_PSI): 2.1 Disposizioni e combinazioni e Compleanni.